

AI-Assisted Microloan Application Readiness Checker

Trade-off Report

Name: Sun Hanyu

Course: PE6201 Emerging AI Technologies

Programme: MSc Enterprise Artificial Intelligence

1. Problem, user, and boundary

I built a pre-check for a Credit Operations Officer who reviews microloan applications before formal credit assessment. My persona handles approximately 40-60 applications per day; this is a scenario assumption, not a measured bank statistic. Actual checking time, rework rate, reviewer wage, and operational savings were not measured.

The system checks required documents, exact numeric inconsistencies, and semantic contradictions between a selected purpose category and free-text explanation. It returns `Complete`, `Missing Documents`, `Inconsistent Information`, or `Manual Review`, with reasons and evidence. It does not assess affordability, score credit, approve or reject a loan, set prices or limits, contact customers, or use external credit data.

Google Cloud Document AI for Lending is the closest documented commercial tool I found. It extracts and classifies lending documents. My prototype addresses a narrower downstream gap: consistency and readiness after fields have already been structured. I do not claim that the commercial product could never be extended to do this.

2. Design and build-versus-buy choice

I use deterministic Python rules for document presence, required fields, invalid values, and exact income comparisons. Gemini 3.8 Flash handles only language interpretation. The final candidate adds bounded retrieval from two project-defined contracts: readiness rules and the human-review contract. These are coursework knowledge, not real bank policy.

The final path is:

```
schema -> rules -> injection guardrail -> retrieval -> one structured Gemini call ->
response and citation validation -> deterministic merger
```

This is RAG rather than an agent because each application needs one constrained judgement, not a variable tool-using loop. I built the synthetic data, labels, rules, retrieval, orchestration, validation, evaluation, guardrails, and Streamlit interface. I rented the foundation-model service through `google-genai`. I did not conduct a low-code comparison. Code was chosen because frozen files, strict schemas, unit tests, retry limits, and leakage controls were central to the assessment. The prototype runs locally but was not deployed to a bank production environment.

3. Data, controls, and evaluation

I used synthetic data to avoid privacy and consent risks. The frozen final set contained 50 cases: 20 clean, five missing-document, five numeric or field contradictions, five ambiguous-purpose, and 15 handwritten semantic contradictions. The 15 scenarios originated from my Chinese examples and were translated and revised with AI assistance.

Applications and labels are separate. Gemini receives only five allowed fields and never receives labels, evaluation slices, or document-presence flags. Tests verify this exclusion; I did not create an intentionally leaky

version merely to manufacture a before-and-after score.

I evaluated a rule baseline, an earlier non-RAG hybrid, and the final RAG hybrid on the same frozen cases. Precision had a 0.70 release floor and recall a 0.90 floor. I also measured manual-review rate, appropriate-review capture, unnecessary review, latency, token use, citations, failures, and cost. Because the final RAG reused the same 50 cases, this is a controlled comparison rather than a new independent holdout. The repository passes 33 offline tests.

4. Results

System	Precision	Recall	F1	TP	FP	TN	FN	Manual review
Rules only	1.000	0.333	0.500	10	0	20	20	0%
Rules + Gemini	0.857	1.000	0.923	30	5	15	0	22%
Rules + project RAG + Gemini	1.000	1.000	1.000	30	0	20	0	10%

The rule baseline missed 20 flag-worthy cases. APP_0286 shows the silent failure: its category was education, but its text said education costs were covered and the loan would buy a motorcycle. Rules returned **Complete**; the final RAG system returned **Inconsistent Information**. The rule output contained no warning that meaning had not been checked.

Final RAG detected all 15 handwritten contradictions and captured all five expected manual-review cases. No clean case was unnecessarily reviewed. All 50 model calls succeeded without retry, all responses cited retrieved context, and all 109 citation identifiers were valid.

One external reviewer independently assessed the 15 handwritten cases. Output agreement was 15/15; 14 citations were supported and one was partially supported because the taxonomy did not clearly define room extensions. This improves the coursework evidence but is not institutional validation.

5. Failure handling, security, and governance

The merger never lets Gemini erase a deterministic finding. Invalid structured output or exhausted provider failure becomes **Manual Review**. In the earlier run, repeated 503 errors for a clean case remained visible as review rather than silently becoming **Complete**.

A deterministic prompt-injection guardrail runs before retrieval and Gemini. Detected attacks skip both steps and become **Manual Review**. The fixed suite blocked six malicious prompts and passed six benign controls. Obfuscated or unfamiliar attacks may still evade this limited pattern-based defence.

I use Singapore's Model AI Governance Framework to support human-centric, explainable, and transparent controls: visible reasons, cited evidence, logged failures, reversible review, and a human checkpoint before credit assessment. I do not claim formal compliance. An officer may request correction, confirm readiness, retain review, or override a readiness label with a recorded reason, but cannot use this tool to approve or reject credit.

6. Cost, accepted trade-off, and limitations

The final run used 47,693 input tokens and an estimated 35,313 output-plus-thinking tokens. At the dated Gemini prices recorded in the project, direct model cost was USD 0.1682 for 50 cases, or USD 0.00336 per application. Average reported model latency was approximately 3.88 seconds. The 10% review rate matters more economically than token price, but real review time and wages were not measured.

I accept provider dependence, added latency, retrieval tokens, direct model cost, and human review in exchange for removing 20 silent false negatives and making sources auditable. I do not accept autonomous credit action.

The evidence remains limited by synthetic reused data, one reviewer, project-defined rather than bank-policy grounding, one partially supported citation, bounded injection tests, no production drift study, and no real deployment. Therefore, the RAG hybrid passes my coursework prototype thresholds; it does not establish production lending readiness.

Performance tuning focused on reducing unnecessary model work rather than enlarging the prompt. I placed deterministic checks and the injection guardrail before retrieval, limited retrieval to two short project contracts, used one structured model call, and rejected invalid schemas or citations. A future version should replace the coursework contracts with approved lender policies, use a genuinely unseen holdout set, obtain additional independent domain review, and measure reviewer time, rework, and drift. Until those steps are completed, I would retain the system as a local pre-check with mandatory human ownership.

References

- Google Cloud, *Document AI for Lending* and *Document AI overview*, accessed 27 September 2026.
- Infocomm Media Development Authority, Singapore, *Model Artificial Intelligence Governance Framework*, Second Edition, 2020.
- PE6201 course materials and project feedback supplied for this assignment.